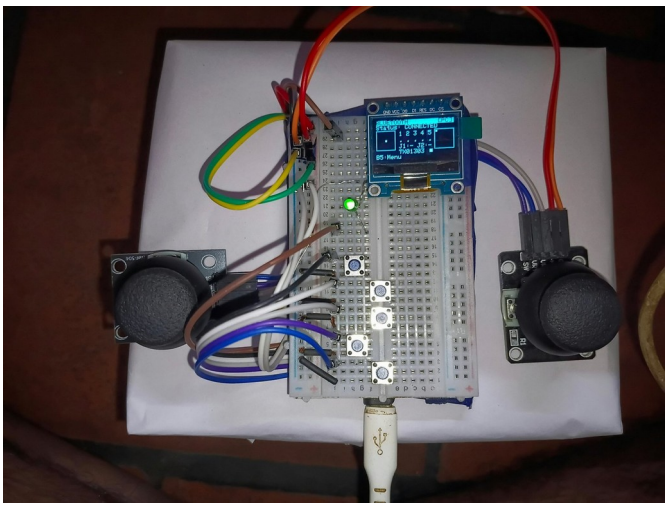


ESP32 Game Controller

Project Documentation

One analog stick and a five-button "+" cluster · Bluetooth gamepad (Xbox / Generic) · USB serial mode · OLED menus · Dino Run, Pac-Man and Snake



The breadboard prototype in Bluetooth mode

Developer	HARRISH YESURAJ P
Firmware version	v2.1 (ESP32_Game_Controller.ino)
Hardware	ESP32 DevKit · SSD1306 OLED · analog stick · 5 push buttons · LED
Date	9 November 2026

Contents

1. Introduction....4
1.1 Purpose of the project....4
1.2 The final design at a glance....4
1.3 How the design evolved....4
1.4 Quick start....5
2. Requirements and Objectives....5
2.1 Functional requirements....5
2.2 Non-functional requirements....5
3. Hardware....6
3.1 Bill of materials....6
3.2 System block diagram....6
3.3 Notes about the ESP32 pins that matter for this project....6
4. Circuit Design....7
4.1 Circuit diagram....7
4.2 Reference project....8
4.3 Power and electrical precautions....9
4.4 Physical layout....9
4.5 The prototype....10
5. Firmware Architecture....11
5.1 Development environment....11
5.2 Structure of the sketch....11
5.3 Data flow....12
5.4 The main loop....12
5.5 Input conditioning....13
5.6 Events and navigation keys....13
5.7 Persistent storage....14
5.8 Boot sequence....14
6. User Interface....14
6.1 Menu structure....14
6.2 Screens....15
6.3 The live view in Bluetooth and USB mode....15
7. Settings Reference....16
7.1 Calibration....16
7.2 Gamepad....17
7.3 Assign buttons....17
7.4 Button positions....17
7.5 Input monitor....18
7.6 Display....18
7.7 LED....18
7.8 Link test....18
7.9 System....18
8. Bluetooth Gamepad....19
8.1 Modes....19
8.2 What is sent....19
8.3 Pairing step by step....20
8.4 Verifying that it works....20
9. USB Mode....21
9.1 Serial protocol....21
9.2 Using the PC helper program....21
9.3 Why the USB mode is not a "plug-and-play" USB gamepad....22
10. Built-in Games....22
10.1 Common behaviour....22
10.2 Dino Run....22

10.3 Pac-Man 22
10.4 Snake 23
10.5 Adding another game 23
11. Animations and Supplied Assets 23
11.1 Game loading animation 24
11.2 Level-up animation 24
12. Testing and Results 25
12.1 Test approach 25
12.2 Test log 25
12.3 Bring-up photographs 25
12.4 The second-stick investigation 26
12.5 Problems found and how they were solved 27
12.6 Offline logic checks 27
13. Troubleshooting Guide 27
14. Safety and Handling 28
15. Version History 28
16. Limitations and Future Work 29
16.1 Known limitations 29
16.2 Ideas for the future 29
Appendix A – Complete GPIO Table 29
Appendix B – Stored Settings (NVS) 30
Appendix C – Target Numbers 30
Appendix D – Delivered Files and Credits 31
Appendix E – Glossary 31

1. Introduction

1.1 Purpose of the project

This project turns an ESP32 development board into a complete, self-contained handheld game controller. It reads one analog stick and five push buttons, shows everything on a small OLED screen, and can present itself to a computer or phone either as a Bluetooth gamepad (an Xbox-compatible controller or a plain HID gamepad) or as a USB serial data source. The same hardware also runs three small games on its own screen, so it can be used with no host connected at all.

The controller was designed, built and debugged step by step on a breadboard. This document describes the finished design (firmware version 2.1): the hardware, the circuit, how the firmware works, how to operate and configure the controller, how it was tested, and what was learned along the way.

1.2 The final design at a glance

Item	Description
Controller	ESP32 DevKit (ESP32-WROOM-32, chip ESP32-D0WD-V3 rev 3.1, 240 MHz dual core, Wi-Fi + Bluetooth), programmed from the Arduino IDE.
Inputs	One analog thumb-stick with a click button (acts as the right stick) and five push buttons arranged as a "+" (Up, Down, Left, Right, Center). By default the "+" is the D-pad and Center is Select.
Display	0.96-inch 128×64 monochrome OLED (SSD1306) on a 4-wire SPI bus.
Feedback	One green LED (status / activity) and the OLED.
Bluetooth	Bluetooth Low Energy HID gamepad in three selectable modes: Xbox (XInput) for PC games, Generic PC and Generic Android.
USB	Serial stream at 115 200 baud plus a small PC program (`usb_host.py`) that turns the stream into a virtual Xbox 360 controller on Windows.
On-board software	Animated boot sequence, Nokia-style main menu, nine settings screens, live calibration, button-assignment tools, an automatic link test, and three games (Dino Run, Pac-Man, Snake) with loading and level-up animations.
Memory	Mapping, calibration, orientation, button positions, display/LED options and high scores are stored in the ESP32 flash (NVS) and survive power cycles.

1.3 How the design evolved

The project went through two main architectures. Understanding why the design changed explains several features (calibration, orientation, wiring tools) and several choices in the final firmware.

Stage	Design	Outcome
Stage 1 (firmware v1.0 – v1.7)	Two analog sticks (J1 and J2) plus five buttons, OLED menus, Bluetooth gamepad, USB serial stream and a Dino game. Later additions: Pac-Man, main menu, games list, animations, calibration and orientation wizards, wiring and pin diagnostics.	Left stick worked well. The second stick (J2) drifted, jumped and appeared to trigger button pins; extensive diagnostics (wiring test, pin test, learn-wiring wizard) showed that the second stick could not be made reliable on the available pins and wiring.
Stage 2 (firmware v2.0 – v2.1)	One analog stick + five buttons in a "+" cluster. The stick acts as the right stick; the "+" gives the D-pad; Center is Select. Bluetooth Xbox (XInput) mode added. Diagnostics tied to the second stick were removed. Snake added in v2.1.	Confirmed working by the developer after firmware v2.0, including the Bluetooth connection to a PC and gameplay.

The Stage 2 control scheme comes from the architecture note supplied with the project (the document "wirinfg_test_1"): the left thumb operates a plus-shaped cluster of five buttons (D-pad directions and Select/Back/View), and the right thumb operates the analog stick, whose click acts as R3.

1.4 Quick start

For readers who only want to get the controller running. Every step is explained in detail later in this document.

1. **Wire it** as in Figure 2 – and power the stick from **3V3**, not 5V (section 4.3).
2. **Upload the firmware** `ESP32_Game_Controller.ino` from the Arduino IDE (section 5.1). The boot logo and the main menu should appear.
3. **Settings → Button positions:** press Up, Down, Left, Right and Center once each, then press any "+" button to save (section 7.4).
4. **Settings → Calibration → Orientation**, then **Calibrate range:** push the stick in each direction, then sweep it to all edges and press Right to save (section 7.1).
5. **Main menu → Bluetooth**, pair "ESP32 Gamepad" / the Xbox entry in the PC's Bluetooth settings, and check the packet counter (section 8.3). Use **Settings → Link test** to verify.
6. Or play a built-in game: **Main menu → Games**.

2. Requirements and Objectives

2.1 Functional requirements

- Read one analog stick (two axes plus a click) and five push buttons reliably, with debouncing, smoothing and a dead zone.
- Present itself to a PC or phone as a gamepad over Bluetooth LE, and be recognised by games without extra drivers (Xbox/XInput mode).
- Offer a USB mode that streams the controller state to a PC program over the USB cable.
- Provide a clear on-device user interface on a 128×64 OLED that can be operated with the controller's own controls only.
- Allow every control to be re-assigned to any game button, with ready-made layouts (D-pad, face buttons, racing).
- Provide calibration, orientation and button-position tools so that the hardware can be assembled in any physical order and still behave correctly.
- Provide a way to **verify** that data really reaches the host (packet counter and an automatic link test).
- Include games that use the same controls, with sound-free feedback through the screen and LED, loading and level-up animations, and saved high scores.

2.2 Non-functional requirements

- **Responsiveness:** button presses are sent within about 5–10 ms; the screen refreshes at about 17 frames per second; games run on a fixed 10 ms simulation step.
- **Robustness:** settings validated on start-up; wizards cannot leave the controller in an unusable state; factory reset always available.
- **Maintainability:** one Arduino sketch with a clearly ordered structure; new games are added to a single table (`GAMES[]`).
- **Safety:** all inputs stay within the 3.3 V limit of the ESP32 pins (see section 4.3).

3. Hardware

3.1 Bill of materials

Qty	Part	Notes
1	ESP32 DevKit (ESP32-WROOM-32, CP2102 USB bridge)	30/38-pin development board with micro-USB. Chip reported by the flasher: ESP32-D0WD-V3, revision v3.1, crystal 40 MHz.
1	Analog joystick module (5-pin: GND, +V, VRx, VRy, SW)	Two potentiometers and a push switch. Power it from 3.3 V.
5	Tactile push buttons	Wired between a GPIO pin and GND; the ESP32 internal pull-up resistors are used.
1	0.96" OLED module, SSD1306, 128×64, SPI (7 pins: GND, VCC, D0, D1, RES, DC, CS)	Driven with the Adafruit SSD1306 library in hardware-SPI mode.
1	Green LED + 1 kΩ resistor	GPIO2 → resistor → LED → GND.
1	Breadboard and jumper wires	Prototype build (see Figure 4).
1	USB data cable (micro-USB)	Power, firmware upload and the USB serial mode. A charge-only cable will not work for uploading.

In the first design a second joystick module was used. It was removed in Stage 2 (section 1.3). No buzzer, battery or additional sensors are used.

3.2 System block diagram

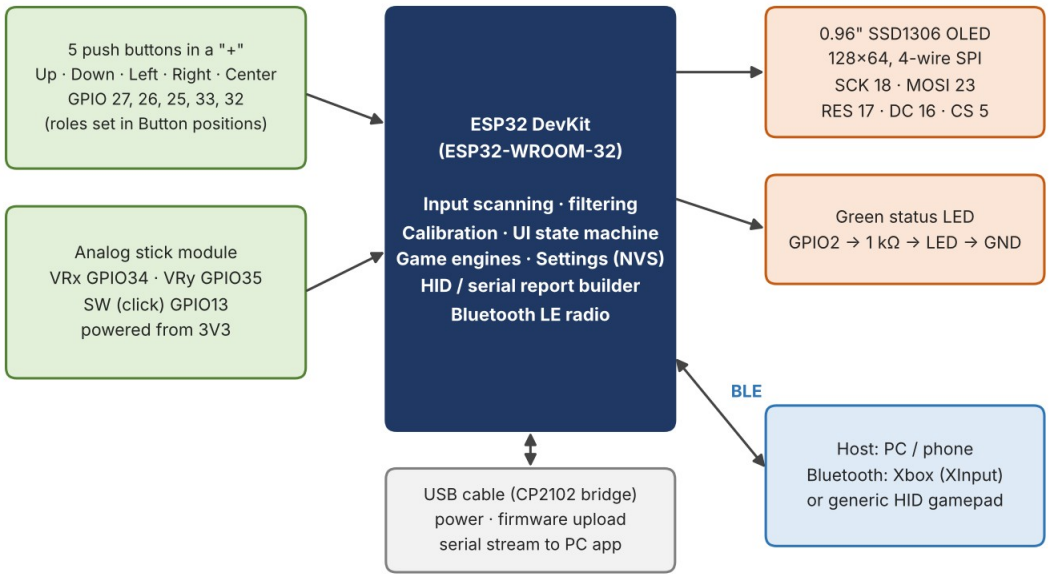


Figure 1: System block diagram: inputs on the left, the ESP32 in the centre, display, LED and host links on the right.

3.3 Notes about the ESP32 pins that matter for this project

- **GPIO34 and GPIO35 are input-only ADC1 pins.** They are used for the stick axes. They have no internal pull-up/pull-down, which is fine for the analog stick. ADC1 can be used while Bluetooth is active.

- **GPIO13, 25, 26, 27, 32 and 33** have internal pull-ups, so a button needs only one wire to the pin and one to GND. A pressed button reads LOW.
- **GPIO2** (the LED) is a boot-strapping pin; it is held LOW at start-up by the firmware, so the LED does not disturb booting.
- **The classic ESP32 has no native USB device.** Its USB port goes to a CP2102 USB-to-serial chip, which is why the USB mode is a serial stream rather than a real USB gamepad (see section 9.3).

4. Circuit Design

4.1 Circuit diagram

Figure 2 is the current circuit diagram supplied with the project. The ESP32 pin header is on the right; five push buttons, the OLED, the analog stick and the LED are on the left.

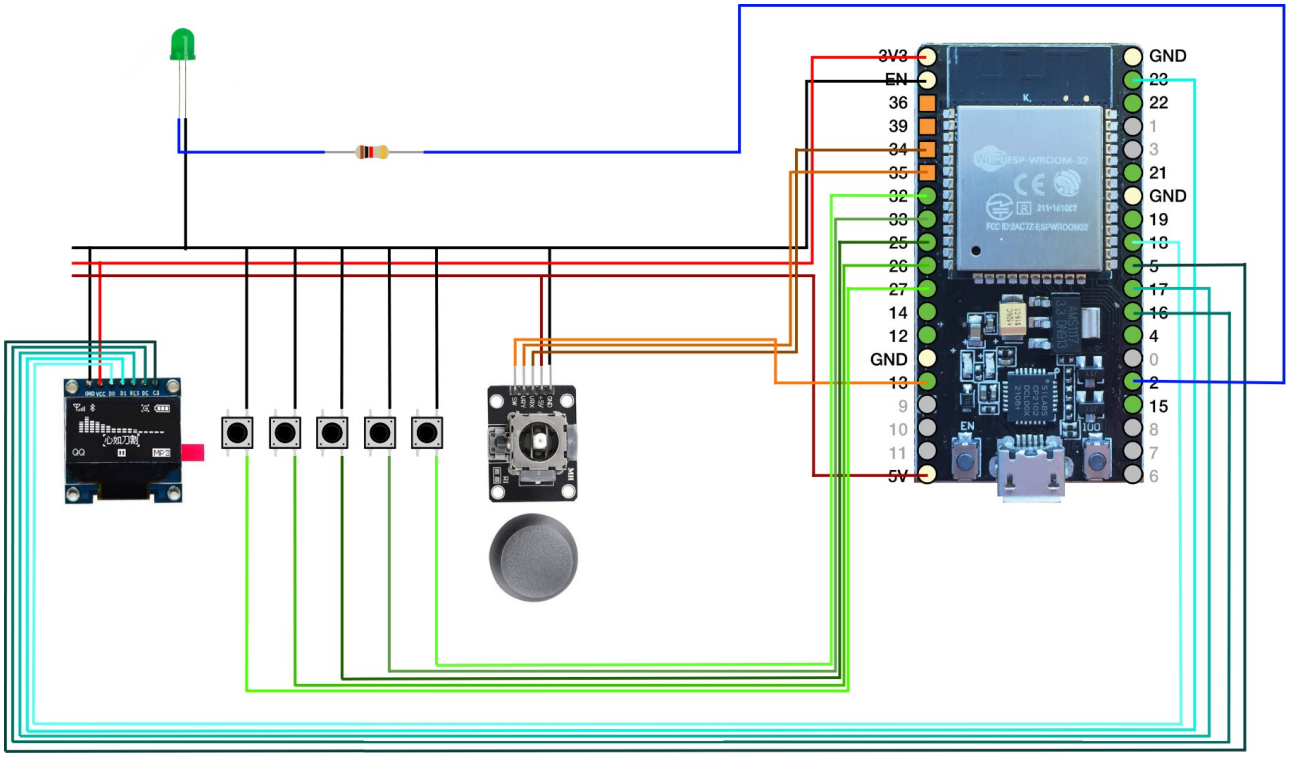


Figure 2: Circuit diagram of the final design (supplied project drawing).

Connection list

Part / pin	ESP32 pin	Notes
OLED GND	GND	Common ground rail.
OLED VCC	3V3	The OLED module is supplied with 3.3 V.
OLED D0 (SCK)	GPIO18	Hardware SPI clock.
OLED D1 (MOSI)	GPIO23	Hardware SPI data.
OLED RES	GPIO17	Reset line (the library pulses it at start-up).
OLED DC	GPIO16	Data / command select.
OLED CS	GPIO5	Chip select.
Stick VRx	GPIO34	Horizontal axis (ADC1 channel 6).
Stick VRy	GPIO35	Vertical axis (ADC1 channel 7).

Part / pin	ESP32 pin	Notes
Stick SW	GPIO13	Stick click, active LOW, internal pull-up.
Stick GND / +V	GND / 3V3	See the warning below.
Button 1 … 5 (left to right in the drawing)	GPIO27, 26, 25, 33, 32	Other leg of every button to GND. Which button is Up, Down, Left, Right or Center is chosen in software (section 7.4).
LED + resistor	GPIO2	GPIO2 → 1 kΩ → LED anode; LED cathode → GND.
USB cable	micro-USB	5 V power, firmware upload, serial stream.

4.2 Reference project

The Bluetooth gamepad idea was developed using a supplied reference project (YouTube channel "Tech at Home", archive `ESP_Bluetooth_Joystick.zip`). It controls a car in a racing game with two analog sticks and a column of buttons. Figure 3 shows its circuit.

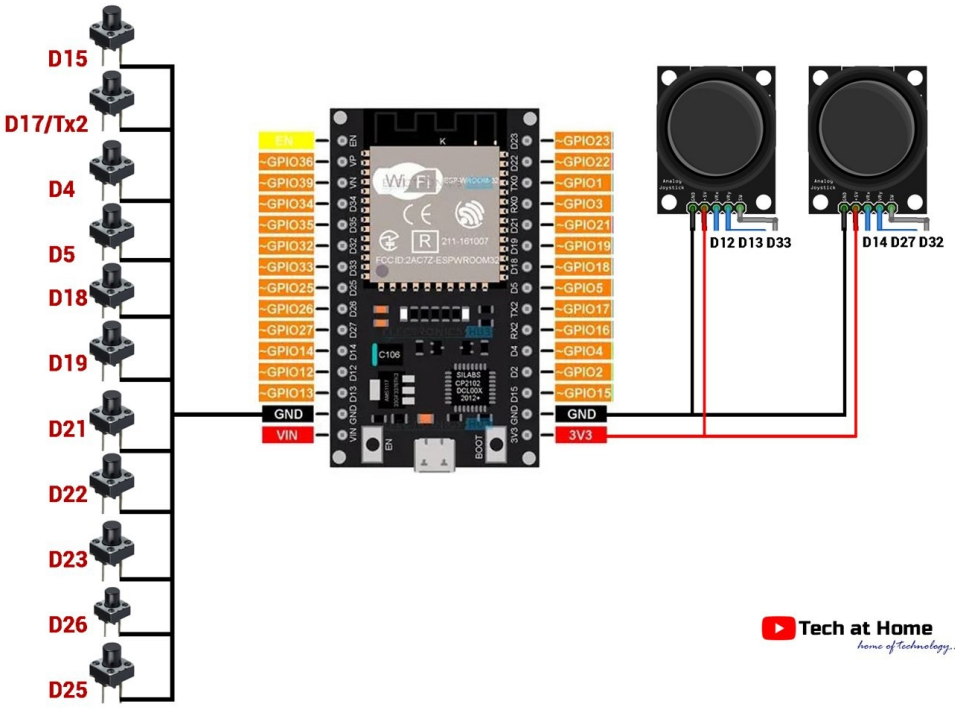


Figure 3: Reference circuit from the supplied ESP_Bluetooth_Joystick project.

What was taken from the reference: the button-numbering tables for different host platforms (PC / Android / PlayStation) and the idea of mapping the right stick to the Z and Rx/Rz axes. What is different here: one stick instead of two, an OLED user interface, runtime calibration, a configurable mapping, Xbox (XInput) mode, USB serial mode and the built-in games. Note that the reference powers both sticks from 3V3.

4.3 Power and electrical precautions

Check the stick supply: use 3V3, not 5V

In the supplied circuit drawing the stick's "+5V" pin is drawn connected to the ESP32 **5V** pin (dark-red wire). The ESP32 analog pins are **not 5 V tolerant**: with a 5 V supply the stick outputs can reach 5 V, which can corrupt the ADC readings of other channels and can damage the pins.

Connect the stick's +V pin to the **3V3** pin (the same rail that feeds the OLED). The firmware helps to catch this mistake: if the stick's resting value reads unusually high (above about 2700 of 4095, which indicates a supply near 5 V) a "!" appears next to Calibration in Settings and the calibration screen shows "5V? Use 3V3!".

- All grounds (OLED, stick, buttons, LED) are common; use the GND pins of the ESP32.
- The LED needs its series resistor (1 kΩ); never connect the LED directly to a GPIO pin.
- Press-buttons are connected between a GPIO pin and GND only. Never connect a button to 3V3 or 5V.
- Keep wires to the stick short and away from the SPI wires if possible; long wires pick up noise on the analog inputs.

4.4 Physical layout

The architecture note describes the intended hand layout. The five buttons are placed as a plus sign under the left thumb; the stick sits under the right thumb:

Control	Position	Default function
Button "Up"	Top of the "+"	D-pad Up
Button "Left"	Left of the "+"	D-pad Left
Button "Center"	Middle of the "+"	Select (View / Back)
Button "Right"	Right of the "+"	D-pad Right
Button "Down"	Bottom of the "+"	D-pad Down
Analog stick	Right thumb	Right stick (X and Y)
Stick click	Press the stick down	R3

Because the five wires are connected to five GPIO pins in the order of the breadboard, the firmware has a small wizard (Settings → Button positions) in which you press Up, Down, Left, Right and Center once. The firmware then remembers which GPIO belongs to which position.

4.5 The prototype

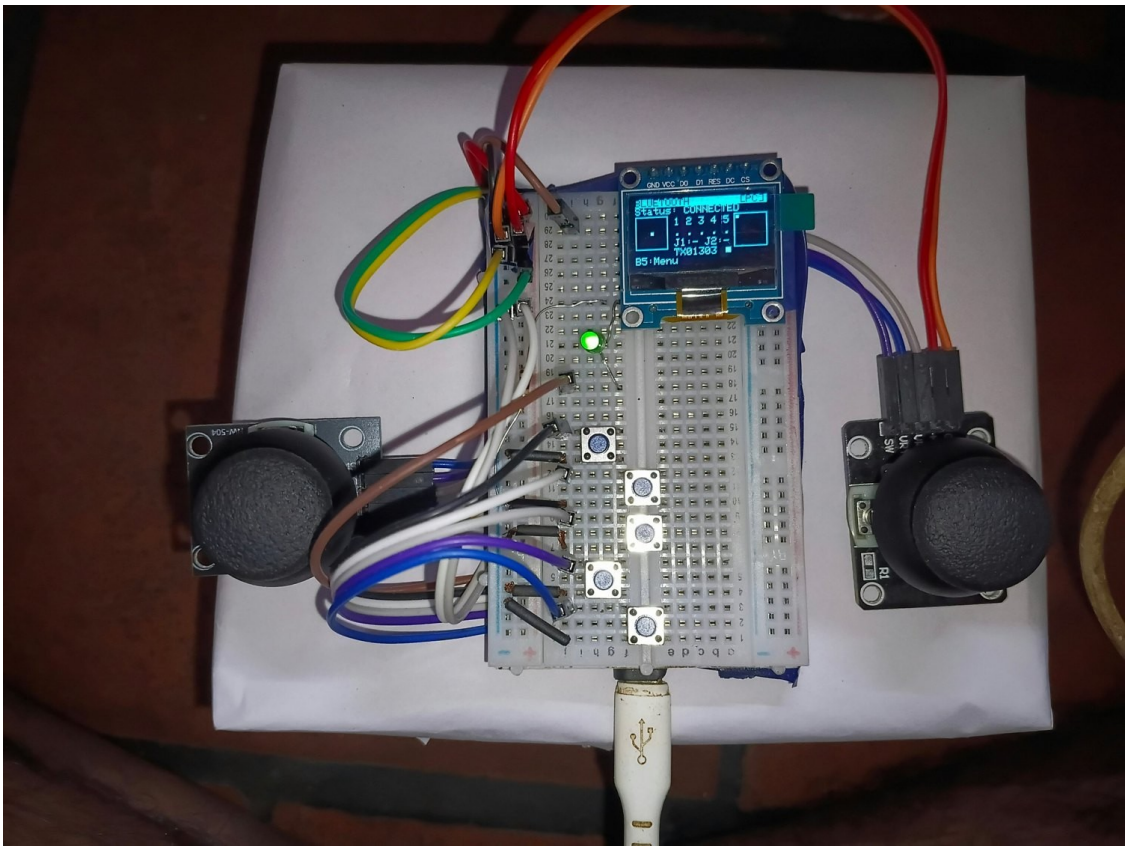


Figure 4: The breadboard prototype: ESP32 and OLED (showing the Bluetooth connected screen with packet counter), the five buttons in the centre, and the analog stick modules. (Photo taken while testing firmware v1.5 with two sticks; the layout of the final one-stick design is described in section 4.4.)

The photograph shows the controller during testing in Bluetooth mode: the OLED reads "BLUETOOTH [PC] Status: CONNECTED", the green LED is lit (solid = connected) and the packet counter (TX01303) shows that reports were being sent to the host.

5. Firmware Architecture

5.1 Development environment

Item	Details
IDE / board	Arduino IDE, board "ESP32 Dev Module" (esp32 core 3.3.x was used for the supplied build logs).
Libraries	Adafruit GFX, Adafruit SSD1306, NimBLE-Arduino, ESP32-BLE-Gamepad 0.8.0 or newer (lemmingDev; needed for the Xbox/XInput mode), Preferences (built in).
Source file	`ESP32_Game_Controller.ino` – one sketch (about 4 400 lines, most of it the animation bitmaps).
Flash use	About half of the default 1.3 MB application partition.
Upload	Serial port at 921 600 baud through the CP2102 bridge (see the upload log: "Hash of data verified").
Compile option	`#define USE_XINPUT 1` – set to 0 if the compiler reports that `GamepadMode` is unknown (older library).

Upload procedure

1. Install the four libraries with the Library Manager (the Bluetooth gamepad library from GitHub as a ZIP if the newest version is not listed).
2. Select the board **ESP32 Dev Module** and the COM port of the CP2102 bridge.
3. Open `ESP32_Game_Controller.ino` and press Upload. If the upload stalls at "Connecting…", hold the BOOT button on the board until it starts.
4. After the upload the board resets by itself and the boot sequence starts. The upload log ends with "Hard resetting via RTS pin…".

> **Harmless compiler notes**
> While compiling, the Arduino IDE prints many lines such as "#pragma message: BT: forcing BR/EDR max sync conn eff to 1". These are informational notes from the Bluetooth library, not errors. The build and the upload still complete successfully.

5.2 Structure of the sketch

The sketch is a single file whose sections appear in the following order (every section only uses things defined above it):

Section	Contents
Config and pins	Timing constants, version, pin numbers, button indices.
Bitmaps	Boot logo (1 024 bytes), loading animation (6 frames), level-up animation (32 frames).
Types and globals	Screen list, event list, settings structure, tables of button names and layouts, game table.
Helpers	Text drawing, boxed menu list with scroll bar, stick box, timer helper.
Settings	Defaults, load/save in flash, validation, upgrade from older versions.
Input	ADC reading and filtering, calibration maths, button debouncing, event generator.
Bluetooth / USB	BLE start-up for Xbox or Generic mode, mask building, report sending, USB serial line, link state.

Section	Contents
Screen control and wizards	Screen switching, calibration, orientation, button-position wizard.
Games	Dino Run, Pac-Man, Snake: logic and drawing.
Drawing	All other screens.
Event handling	One handler per screen, LED control.
Boot and main loop	Start-up sequence, `setup()`, `loop()`.

5.3 Data flow

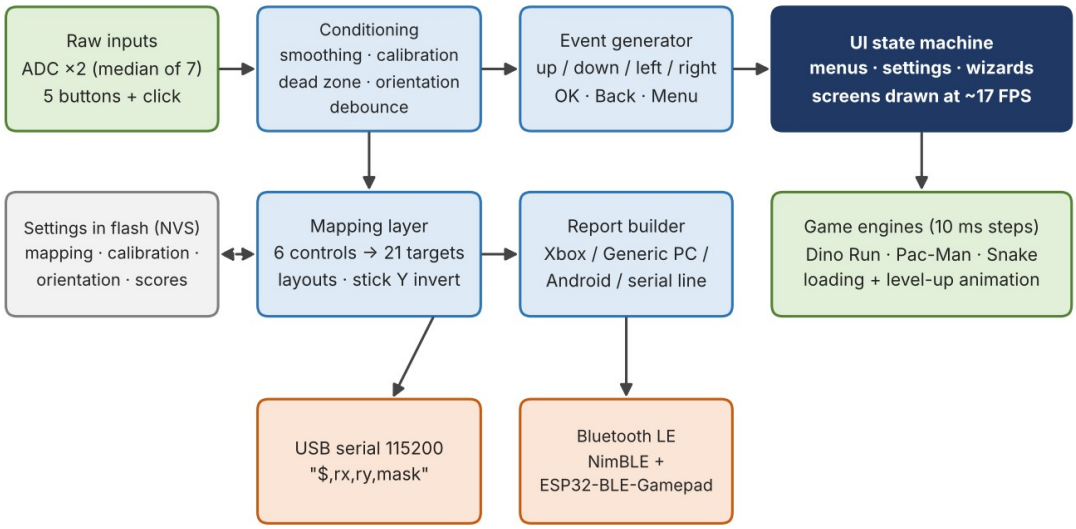


Figure 5: Firmware data flow from the raw inputs to the host and to the games.

5.4 The main loop

`loop()` runs continuously and does the same steps every pass (about every millisecond):

1. Read the two ADC channels and the six digital inputs (`readInputs`).
2. Turn them into events: up, down, left, right, OK, back, menu (`genEvents`).
3. Update the Bluetooth connection state or read the serial port (`updateLink`).
4. Wake the screen if it was sleeping (the first input only wakes it).
5. Hand each event to the handler of the current screen (`handleEvent`).
6. Run the per-screen work: wizards, game input, loading / level-up timers.
7. Send the gamepad report or serial line to the host (`sendHost`) and update the LED.
8. Sleep the screen after the configured timeout (never during games, calibration, tests or animations).
9. Advance the running game in fixed 10 ms steps, then redraw the screen about every 59 ms (about 17 frames per second).

> **Timing detail worth knowing**
>
> Several screen changes store the current time. To avoid wrong "negative" times inside one loop pass, all elapsed-time checks use a small helper (`since()`) that never returns a negative value. An early version without it made the screen go to sleep right after a button press and made the loading animation vanish; this bug was found and fixed during testing (section 12).

5.5 Input conditioning

Analog stick

- **Sampling:** 12-bit ADC with 11 dB attenuation. For each reading the channel is read three times and discarded (to flush the ADC multiplexer), then seven readings are taken and the **median** is used. This removes the single-sample spikes of the ESP32 ADC.
- **Smoothing:** exponential filter, new = old + 0.30 × (reading − old).
- **Calibration:** each channel has a minimum, a centre and a maximum. A reading is converted to −1 … +1 by dividing by the distance to the centre on the relevant side, then a **12 % dead zone** is removed and the rest is rescaled:

```
if raw >= centre:  v =  (raw - centre) / (max - centre)
else:              v = -(centre - raw) / (centre - min)
if |v| < 0.12:     v = 0
else:              v = sign(v) * (|v| - 0.12) / 0.88
```

- **Orientation:** the two logical axes (X, Y) can be taken from either ADC channel with either sign, so a stick can be mounted rotated or reversed without changing the code. Convention used everywhere: right = +, down = + (up = −).
- **Auto-centre at boot:** if the stick rests within 800 counts of the saved centre, the centre is refreshed from the live value (keep the stick released while the controller starts).

Buttons

- Active LOW with internal pull-ups.
- **Debounce:** a change is accepted immediately and then ignored for 20 ms (no lost taps).
- Pressing and releasing generate edge flags used by the event generator.

5.6 Events and navigation keys

All menus are driven by seven events. The same physical control produces different events depending on the screen:

Control	Menus and lists	Value screens	Link screens (Bluetooth / USB)
Up / Down button	Move the cursor	Move between rows / change value	Sent to the host
Left button	Back	Decrease / previous value	Sent to the host
Right button	Open / OK	Increase / next value	Sent to the host
Center button	Back (pause during a game)	Back (saves the settings)	Sent to the host; hold 1.2 s = main menu
Stick click	OK	OK / cycle	Sent to the host (R3 by default)
Stick	Moves the cursor (0.6 on / 0.4 off, repeat 400 ms then 150 ms)	Left/right changes values	Sent to the host

"Menus and lists" are the main menu, games list, settings list, calibration menu, assign-buttons list, system menu and the start / pause / game-over screens of the games (Right = OK, Left = Back). "Value screens" are Display, LED, Gamepad and the assign-button editor, where Left and Right change the value. During calibration, orientation and button-position wizards the stick does not move the cursor.

5.7 Persistent storage

Settings live in the flash through the ESP32 `Preferences` (NVS) library, namespace `ctrl`. Every value is validated when it is loaded; invalid or missing values fall back to defaults. A settings version number allows upgrades: when the firmware finds an older version it replaces the mapping with the new defaults but keeps calibration and high scores. A complete key list is in Appendix B.

5.8 Boot sequence

After power-up or reset the OLED shows a short animated sequence while the firmware performs real checks:

#	Stage	What happens
1	Power-up (0.15 s)	Screen black; settings are loaded from flash.
2	Logo (1.0 s)	The logo of Figure 6 appears and the contrast fades up over 0.4 s.
3	Splash (1.15 s)	"ESP32 – Game controller – Stick + 5 buttons" and the version.
4	Loading (4 × 0.35 s)	Four lines with a progress bar: Display, Settings, Inputs, Radio. The Inputs step measures the stick centre and flags a stuck or too-high reading; the Radio step is a placeholder because Bluetooth starts only when you open it from the menu.
5	Ready (0.4 s)	READY with a full bar; the LED blinks once. Then the main menu opens.



Figure 6: The boot logo stored in the firmware (128×64 pixels, shown enlarged).

If the OLED cannot be started (wrong SPI wiring), the firmware prints an error on the serial port and blinks the LED at 5 Hz forever, so the problem is visible even without a screen.

6. User Interface

6.1 Menu structure

After the boot sequence the main menu opens. It is drawn in the style of an old mobile phone: one framed list with a vertical scroll bar on its right side. From the main menu everything is reachable.

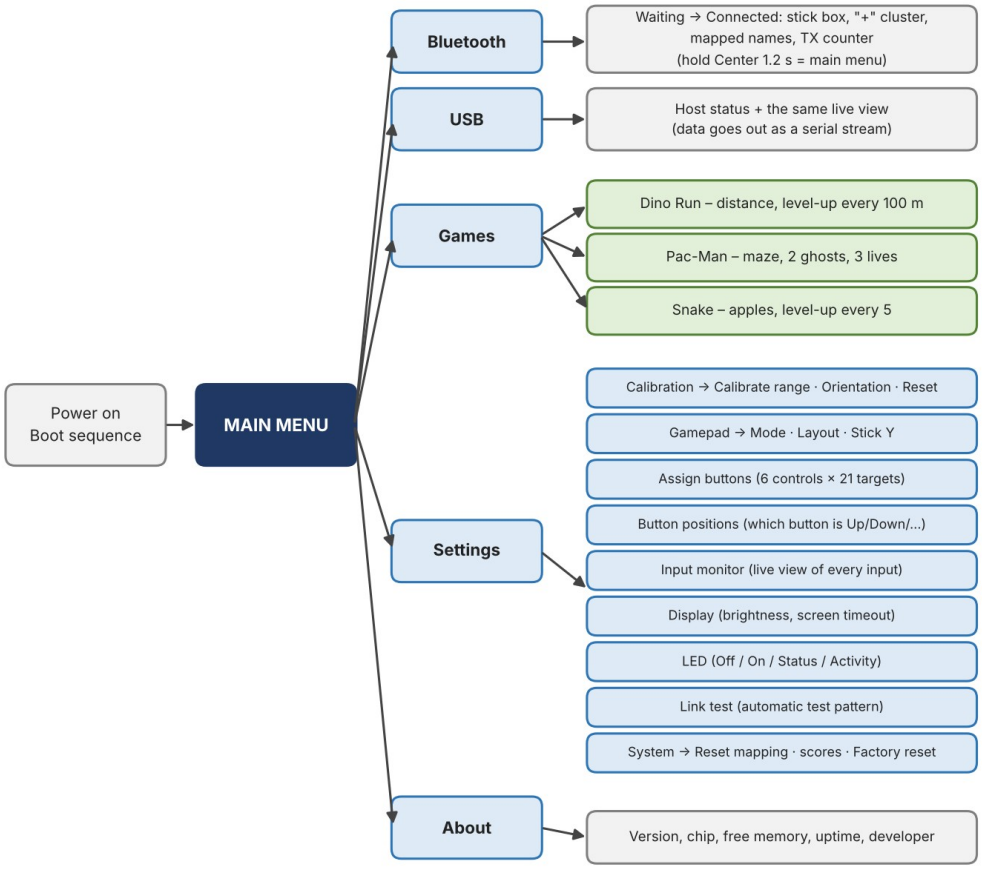


Figure 7: Menu structure of firmware v2.1.

6.2 Screens

Screen	What it shows	Leave with
Main menu	Bluetooth, USB, Games, Settings, About.	Left / Center (returns to the previous screen)
Bluetooth	Waiting screen with pairing hints, then the live view (section 6.3).	Hold Center 1.2 s
USB	Host status and the same live view; the data goes out as a serial stream.	Hold Center 1.2 s
Games	List of games with the best score of each, loading animation when one is opened.	Left / Center
Settings	Nine entries (section 7). A "!" next to Calibration means calibration or orientation is missing or the stick supply looks wrong.	Left / Center
About	Firmware version, chip model, free heap memory, uptime, developer name.	Center
Confirmation	Yes/No question before any reset; the factory reset needs the Yes button held for 2 s.	Left / Center

6.3 The live view in Bluetooth and USB mode

While a link is active the screen is a live picture of the controller and shows what is being sent:

- **Status row:** "CONNECTED" (Bluetooth) or "Host: CONNECTED / waiting" (USB), and the platform tag in the title bar (`XB` Xbox, `PC`, `AND` Android, or `USB`).
- **The "+" cluster:** five small squares in a plus shape. Each square shows the short name of the game button assigned to that control (A, B, X, Y, LB, RB, LT, RT, Se, St, L3, R3, ^ v < > for the D-pad, L^ Lv L< L> for left-stick directions). A pressed button is drawn filled.
- **The stick box:** a square with a dot that follows the stick. Under it the name of the stick-click assignment and its state.
- **TX counter:** the number of reports sent so far, with a small square that flashes on every packet. If the counter does not move, nothing is being sent.
- **Footer:** "Hold Center: menu" – the Center button is a normal game button here, so it must be held for 1.2 s (a progress bar appears) to return to the menu.
- **LED:** blinking once per second = waiting; solid = connected.

7. Settings Reference

Settings are reached from Main menu → Settings. The nine entries are described below. Changes are saved to flash when you leave the screen (or immediately, where noted).

7.1 Calibration

The submenu has three entries: Calibrate range, Orientation and Reset calibration. Calibration is needed because no two potentiometers have exactly the same centre and travel.

Calibrate range

1. Open Settings → Calibration → Calibrate range. The controller measures the resting position for about 0.4 s – keep the stick released.
2. Sweep the stick slowly to all four edges. The box on the right of the screen shows the stick; two percentages show how much of the expected travel has been covered on each axis.
3. If the resting position looks wrong, release the stick and press **Up** (set centre). **Down** restarts the range measurement.
4. Press **Right** to save. If an axis covered less than 50 % of its travel, the first press only shows "Low range!"; press Right again to save anyway.
5. **Center** cancels without saving.

> **What the screen tells you**
> The third text line shows the raw ADC values of both axes. If the stick centre reads very high (above about 2700) the screen shows "5V? Use 3V3!" instead – check the stick's supply wire (section 4.3).

Orientation

The wizard asks you to push the stick Up, Down, Left and Right, one after another, releasing it in between. It detects which ADC channel moved and in which direction, checks that opposite pushes agree (Up and Down must be the same channel with opposite signs, Left and Right likewise) and that X and Y are different channels. If everything is consistent the orientation is saved; otherwise the screen explains what disagreed ("UP/DOWN differ", "LEFT/RIGHT differ", "X and Y same wire") and you can retry. Use this wizard whenever the stick feels reversed or swapped.

Reset calibration

After confirmation the saved calibration and orientation are erased and the stick is re-centred with default ranges. The "!" marker reappears until calibration is done again.

7.2 Gamepad

Three rows, selected with Up / Down and changed with Left / Right (the stick click also steps to the next value):

Row	Values	Meaning
Mode	Xbox · Generic PC · Android	How the controller presents itself to the host (section 8.1). Changing between Xbox and the generic modes restarts the board and requires pairing again.
Layout	D-pad · Face btns · Racing · Custom	Ready-made assignments of the six controls (section 7.3). "Custom" is shown automatically when the current assignment matches none of them.
Stick Y	Normal · Inverted	Inverts the vertical axis of the right stick for the game (for example if the camera looks the wrong way).

Leaving the screen with Center saves the settings. If the mode changed between Xbox and generic, the controller shows "Restarting…" and reboots by itself.

7.3 Assign buttons

The list shows the six physical controls (Up, Down, Left, Right, Center, Click) and the game button each one currently sends. Open one with Right, pick a new target with Left / Right (or Up / Down) and save with the stick click or Center. The 21 possible targets are:

Group	Targets
Face and shoulder	A, B, X, Y, LB, RB, LT, RT
System	Select (View), Start (Menu), L3, R3
D-pad	Pad Up, Pad Down, Pad Left, Pad Right
Left stick	LS Up, LS Down, LS Left, LS Right (a button press moves the left stick to full deflection)
Other	None (the control sends nothing)

Three ready-made layouts are available in Settings → Gamepad → Layout. The stick click is R3 and Center is Select in all of them:

Layout	Up	Down	Left	Right	Typical use
D-pad	Pad Up	Pad Down	Pad Left	Pad Right	Menus, platformers, general use (default)
Face btns	Y	A	X	B	Games that need the four action buttons
Racing	RT (gas)	LT (brake)	LS Left (steer)	LS Right (steer)	Racing games; the stick becomes the right stick for the camera

7.4 Button positions

The five buttons are wired to GPIO 27, 26, 25, 33 and 32, but nothing forces the left-most button to be "Up". This wizard asks you to press Up, Down, Left, Right and Center once each (it waits for the buttons to be released between presses). When all five are learned the screen lists the GPIO numbers.

- **Press any of the five "+" buttons to save** the new order.

- **Click the stick to cancel** (the old order stays). During the wizard, holding the stick click for 2 s also cancels.

The wizard only accepts the five known button pins and never uses the same pin twice, so a wrong order cannot disable a button. (In version 2.0 the confirmation relied on the stick click, which did not work on that screen; this was corrected in version 2.1 – see section 15.)

7.5 Input monitor

A live test screen: the "+" cluster with assigned names, a box showing the stick position, the stick-click state and the raw ADC numbers of both axes. It is the quickest way to check that every button and the stick respond. The screen never sleeps while open.

7.6 Display

- **Brightness:** five levels (OLED contrast 26, 64, 128, 191, 255); applied immediately.
- **Timeout:** Off, 30 s, 1 min, 5 min. After the timeout the screen is switched off; the first input only wakes it and is not used for anything else. The screen never sleeps during games, wizards, animations and tests.

7.7 LED

Mode	Behaviour
Off	The LED is never lit (it also stays off during boot).
On	Always lit.
Status	Bluetooth or USB: blinks once per second while waiting, solid when connected. Games: two quick blinks at game over. Boot: lit during loading and blinks once at READY.
Activity	Flashes briefly (50 ms) at every button press or stick movement.

7.8 Link test

The link test answers the question "is the controller really sending the right signals?" without touching any control. Open it after connecting Bluetooth (or opening USB): the controller sends an automatic pattern – the stick circles around, and the 20 game buttons are sent one after another every half second – while the screen shows the name of the button being sent and the packet counter. Watch a gamepad tester page on the host: if it shows the same pattern, the whole path works. If the link is not connected the screen says so.

7.9 System

Entry	Effect
Reset mapping	Restores the default assignment (D-pad layout).
Reset high scores	Sets the best scores of all three games to zero.
Factory reset	Erases every saved setting and restarts. Select Yes and hold the stick click (or the Right button) for 2 s; a progress bar is shown.

8. Bluetooth Gamepad

8.1 Modes

The Bluetooth radio is started the first time you open Main menu → Bluetooth (not at power-up). The controller is a Bluetooth Low Energy HID device. The way it describes itself to the host (the "HID report descriptor") is fixed when the radio starts, which is why switching between Xbox and generic modes needs a restart.

Mode	What the host sees	Best for
Xbox (XInput)	An Xbox controller (Series X profile of the ESP32-BLE-Gamepad library): 11 buttons, two analog triggers, one hat (D-pad), two sticks. Native XInput over Bluetooth.	PC games (Windows), Android, Steam and anything that expects an Xbox controller. Default.
Generic PC	A plain HID gamepad (vendor id 0xE502, product id 0xABCD): 15 buttons, one hat, axes X, Y, Z, Rx, Rz.	Web gamepad testers, older applications.
Android	The same plain HID gamepad, with button numbers arranged as Android expects.	Android phones/tablets whose games use generic controllers.

8.2 What is sent

Button numbers

Every target of the mapping is translated into the number the chosen mode needs. LT and RT are analog triggers in Xbox mode (a press sends full travel) and ordinary buttons in the generic modes.

Target	Xbox	Generic PC	Android	Remark
A	1	1	1	
B	2	2	2	
X	3	3	4	
Y	4	4	5	
LB	5	5	7	Left shoulder
RB	6	6	8	Right shoulder
LT	analog trigger	7	9	Left trigger
RT	analog trigger	8	10	Right trigger
Select	9	9	11	View / Back
Start	10	10	12	Menu
L3	7	11	14	Left stick click
R3	8	12	15	Right stick click
Pad Up/Down/Left/Right	hat	hat	hat	Eight-way hat switch (diagonals are formed automatically)
LS Up/Down/Left/Right	left stick	X, Y axes	X, Y axes	Full deflection while pressed

Sticks

Mode	Left stick	Right stick (the physical stick)
Xbox	From "LS" targets (±32 767)	Right thumb X/Y, ±32 767 in 63 steps per side
Generic PC	Axes X and Y (0 – 32 767, centre 16 383)	X on axis Z, Y on axis Rx
Android	Axes X and Y	X on axis Z, Y on axis Rz

Report timing

- A **button change** is sent at once (at least 5 ms after the previous report).
- A **stick-only change** is sent at most 40 times per second (every 25 ms). The stick value is quantised to 63 steps per side, so small noise never creates a report.
- While nothing changes a **keep-alive** report is sent every 100 ms, so a host that subscribes late still receives the current state.
- When the connection is made, or the layout / mode changes, a full report is forced; when you leave the Bluetooth screen, one neutral report (everything released) is sent so no button stays pressed on the host.

8.3 Pairing step by step

1. On the controller open **Main menu → Bluetooth**. The screen shows "Waiting for connection" and, in generic modes, the name "ESP32 Gamepad".
2. On the PC open Settings → Bluetooth & devices → Add device → Bluetooth. Select the controller from the list (in generic modes it is named "ESP32 Gamepad"; in Xbox mode the host may show it as an Xbox-style controller). On Android use Settings → Connected devices → Pair new device.
3. When pairing completes the controller screen changes to **CONNECTED**, the LED turns solid and the TX counter starts counting.
4. Test it (section 8.4).

After every firmware change that touches the Bluetooth settings: forget the device and pair again

Hosts remember the description of the device from the first pairing. If the controller shows "CONNECTED" but the tester or game sees nothing, remove the controller ("ESP32 Gamepad" or the Xbox-style entry) from the host's Bluetooth list and pair again. The same applies after switching between Xbox and generic mode.

8.4 Verifying that it works

- **On the controller:** the TX counter must increase and its square must flash while you press buttons.
- **Link test:** Settings → Link test sends a known pattern; compare it with what the host shows.
- **On the host:** open a gamepad tester web page (browsers only list a gamepad after the first button press, so press a button). On Windows, "Set up USB game controllers" (`joy.cpl`) shows the buttons and axes live.
- **In a game:** Xbox mode is recognised as a normal Xbox controller. For games that do not accept the controller, Steam can map any gamepad (Steam Input).

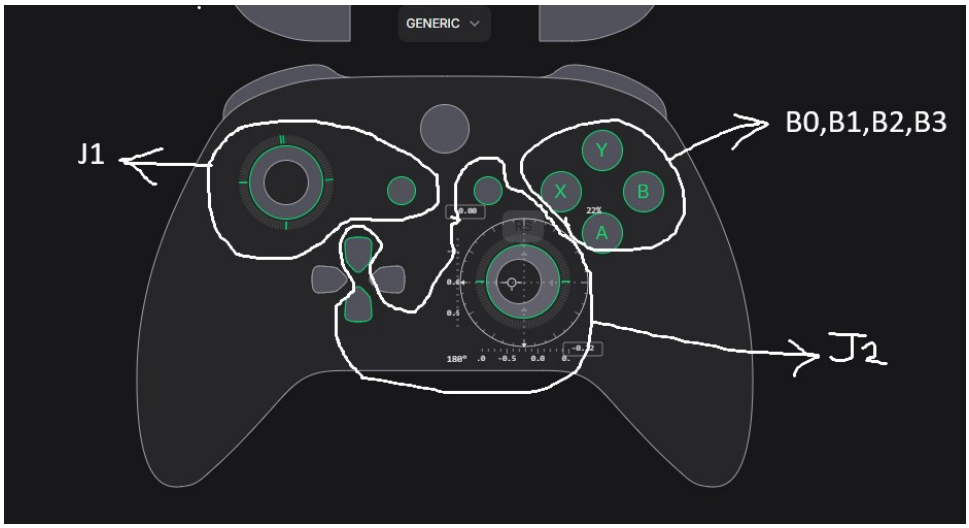


Figure 8: A gamepad tester showing the Xbox layout used as the target layout for this project (face buttons B0–B3, the two sticks). The annotations come from the design stage.

9. USB Mode

9.1 Serial protocol

In USB mode the controller sends one text line per update over the USB cable at 115 200 baud. The line has the form:

```
$,rx,ry,mask
```

Field	Meaning
rx, ry	Right stick, −1000 … +1000 (up is negative).
mask	Decimal number; bit (n−1) is set while game button n is pressed: 1 A, 2 B, 3 X, 4 Y, 5 LB, 6 RB, 7 LT, 8 RT, 9 Select, 10 Start, 11 L3, 12 R3, 13–16 D-pad Up/Down/Left/Right, 17–20 left-stick Up/Down/Left/Right.

Lines are sent whenever something changes (at most every 10 ms) and every 500 ms as a heartbeat. Any byte received from the PC marks the host as "connected"; the screen shows "Host: waiting…" until then.

9.2 Using the PC helper program

1. Install Python and run `pip install pyserial vgamepad` (vgamepad is Windows only).
2. Close the Arduino Serial Monitor (only one program can use the port).
3. On the controller choose **Main menu → USB**.
4. Run `python usb_host.py COM3` (use your port name). The program sends a heartbeat every second and creates a **virtual Xbox 360 controller** driven by the incoming lines: right stick, buttons, triggers, D-pad and left stick.
5. Without vgamepad the program simply prints the values, which is also a good way to check the data.

Without the helper you can open the Arduino Serial Monitor at 115 200 baud, type any character, and watch the lines scroll by while you press buttons.

9.3 Why the USB mode is not a "plug-and-play" USB gamepad

The classic ESP32 used in this project has no USB device controller; its micro-USB socket is wired to a CP2102 USB-to-serial chip. A computer therefore sees a serial port, not a gamepad. A true plug-and-play USB gamepad needs an ESP32-S2 or ESP32-S3 board (native USB with the Arduino USB HID gamepad library). That is listed under future work (section 16). Bluetooth already provides a driver-free gamepad for PCs and phones.

10. Built-in Games

10.1 Common behaviour

- Open **Main menu → Games**, pick a game and press Right (or the stick click). A **loading animation** (six frames, 0.72 s – always at least 0.6 s) plays, then the game's start screen shows the best score.
- Start screen: Right / stick click = start; Left / Center = back to the games list.
- During play: the "+" buttons or the stick steer; **Center** pauses. Paused: Right = resume, Left / Center = back to the games list.
- When a level is completed the **level-up animation** plays (32 frames, about 1.8 s); Right / stick click skips it.
- Game over: two quick LED blinks; the score and best score are shown; Right = retry. A new best score is stored in flash and shown in the games list.
- All games use the same fixed 10 ms simulation step, so their speed does not depend on how fast the screen is redrawn.

10.2 Dino Run

An endless runner: a dinosaur runs to the right over desert obstacles.

Item	Details
Controls	Jump: Up button (or stick up). Duck: Down button (or stick down); holding Down in the air makes you fall faster.
Obstacles	Small cactus, tall cactus, double cactus, and (from level 2) flying birds that must be ducked under or jumped over.
Score	Distance in metres (12 pixels = 1 m). The best distance is saved.
Levels	Every 100 m: level-up animation, the track is cleared and the speed increases. Speed = 60 + 8 × level pixels per second (level capped at 10, i.e. 140 px/s).
Jump physics	Gravity 500 px/s², launch speed 190 px/s: a jump is about 36 pixels high and lasts about 0.76 s – long enough to clear a double cactus even at slow speed. (In simulation the timing window for a double cactus is 300–440 ms.)

10.3 Pac-Man

A small maze game on a 21 × 9 tile board (6-pixel tiles).

Item	Details
Controls	Any of the four direction buttons or the stick. The wanted direction is remembered, so you can press a turn early; reversing is instant.
Scoring	Dot 10, power pellet 50, ghost eaten 200.

Item	Details
Ghosts	Two ghosts leave their corners after 1.5 s and 3.5 s. They chase the player (80 % of the time they choose the direction that reduces the distance, 20 % random) and never turn back on themselves.
Power pellets	Ghosts turn hollow and can be eaten for 6 s (shortened by 0.4 s per level, minimum 2.5 s); they flash before the effect ends.
Lives	Three. Touching a ghost loses a life and everything is reset for 1.5 s.
Levels	Clearing every dot plays the level-up animation and starts the next maze. Speeds rise with the level: Pac-Man 5.0 tiles/s (+0.2 per level, max 6.0), ghosts 3.4 tiles/s (+0.25 per level, max 4.8), frightened ghosts 2.8 tiles/s.

10.4 Snake

The classic snake game on a 31 × 13 grid of 4-pixel cells, inside a frame.

Item	Details
Controls	Direction buttons or stick. A 180° turn into the snake's own body is ignored.
Scoring	10 points per apple; the snake grows by one cell.
Levels	Every 5 apples: level-up animation and a faster snake. The step time is 170 ms − 9 ms × level (minimum 80 ms).
Game over	Hitting the wall or the snake's own body (the tail cell is free when the snake is not growing).

10.5 Adding another game

The games list is built from a table at the top of the sketch. To add a game:

1. Add screens for it to the `Scr` list (start, play, pause, over), a best-score field to `Config` with load/save, and write its logic and drawing functions.
2. Add one line to `GAMES[]` (name, start screen, pointer to its best score).
3. Add the screens to the helper lists (`listScreen`, `stickNavScreen`, `clickOkScreen` – the play screen belongs in the "no menu navigation" lists), the sleep exceptions in `loop()`, the simulation step and the game-over LED blink.
4. Add the event cases (start, pause, game over) next to the existing games. Snake is the shortest complete example to copy.

```
const GameDef GAMES[] = {
  { "Dino Run", SC_DINO_START,   &cfg.hi  },
  { "Pac-Man",  SC_PAC_START,    &cfg.phi },
  { "Snake",    SC_SNAKE_START,  &cfg.shi },
};
```

11. Animations and Supplied Assets

Two animations supplied by the developer are built into the firmware as 128×64 one-bit bitmaps stored in flash and drawn with `drawBitmap`.

11.1 Game loading animation

Source: the "Super Saiyan" display sketch (six frames, 120 ms each = 0.72 s). It plays every time a game is opened from the list and is always shown for at least 0.6 s, as requested.



Figure 9: The six loading-animation frames (shown at twice the size).

11.2 Level-up animation

Source: `goku-dragon-ball-level-up-anime-gif.h` (32 frames at 55 ms ≈ 1.8 s). It plays in every game when the next level is reached. The figure shows eight of the 32 frames.



Figure 10: Every fourth frame of the 32-frame level-up animation.

Notes on the supplied files

The original header file used hyphens inside the array names, which are not valid C identifiers; the frames were copied into the sketch under valid names (`lvl_f0 … lvl_f31`, `ld_f0 … ld_f5`).

The sixth frame of the loading animation contained text accidentally pasted into the middle of the data ("SYSTEM … About"). It was repaired by replacing the damaged rows with neighbouring rows, so that frame may differ very slightly from the original picture. Re-exporting that frame will restore it exactly.

12. Testing and Results

12.1 Test approach

The controller was tested in three ways: (1) on the bench, reading the OLED screens and the LED; (2) against real hosts, with web gamepad testers, a PC and games; and (3) offline, by running the firmware logic on a computer with simulated inputs to check things that are hard to trigger by hand (wizards, report building, game rules). Results below combine the developer's observations with those checks.

12.2 Test log

ID	Test	Observation / result	Status
T1	OLED start-up and SPI wiring	The boot logo, splash, loading steps and menus appear correctly (Figures 11 and 12 show screens from the first power-up).	Pass
T2	Input monitor – raw ADC values	Left stick centre values were stable. The second stick reported impossible values (for example 0 on one axis), revealing a hardware/wiring fault.	Pass (found fault)
T3	First calibration	Failed with centres near 2950 instead of about 2000, because the stick was fed from 5 V and the first limits were strict. Led to the 3V3 warning and the simplified calibration.	Fixed
T4	Bluetooth pairing and gamepad tester	Connected and detected in a web gamepad tester; the OLED showed CONNECTED and a rising packet counter.	Pass
T5	Bluetooth after firmware changes	Once connected but not detected: the host had cached the old device description. Removing the device and pairing again fixed it.	Pass (procedure documented)
T6	USB serial stream with the PC helper	Worked. The tester showed the second stick stuck at −1.00, confirming the second-stick fault (Figure 13).	Pass (found fault)
T7	Button mapping and button order	Assignments work; saving a new button order failed in v2.0 (the confirmation key did not work on that screen). Fixed in v2.1.	Fixed
T8	Final one-stick design with Xbox mode	Reported working by the developer after firmware v2.0 (Bluetooth connection, controls and gameplay).	Pass
T9	Dino Run	Works. Jumping over a double cactus was too hard at slow speed; jump physics retuned.	Pass (tuned)
T10	Pac-Man, Snake	Pac-Man (v1.1) and Snake (v2.1): rules verified by offline simulation (maze clearing and level-up, ghosts; snake growth, scoring, level-up after 5 apples, wall and self collision). Play-testing on the device continues.	Pass (offline) / on-device pending

12.3 Bring-up photographs

Figures 11 and 12 were taken on the first day (firmware v1.0, two-stick version). The text is rotated in the originals because the board was lying sideways; the pictures below are rotated back.

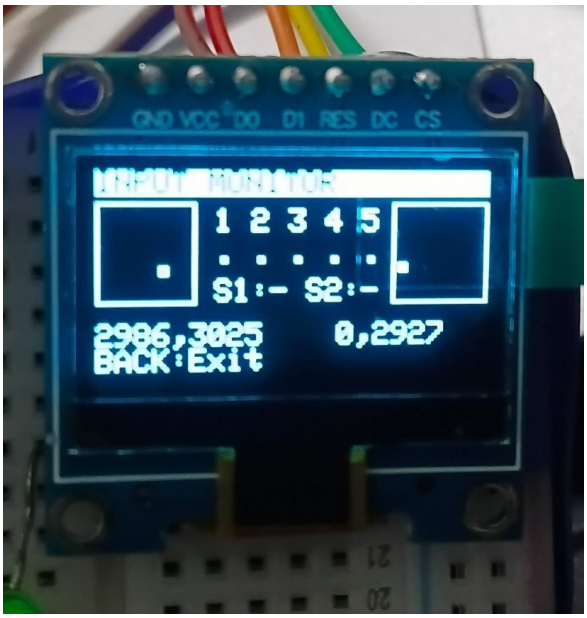


Figure 11: Input monitor screen: stick boxes, button states and raw ADC values (the second stick reads 0 on one axis).

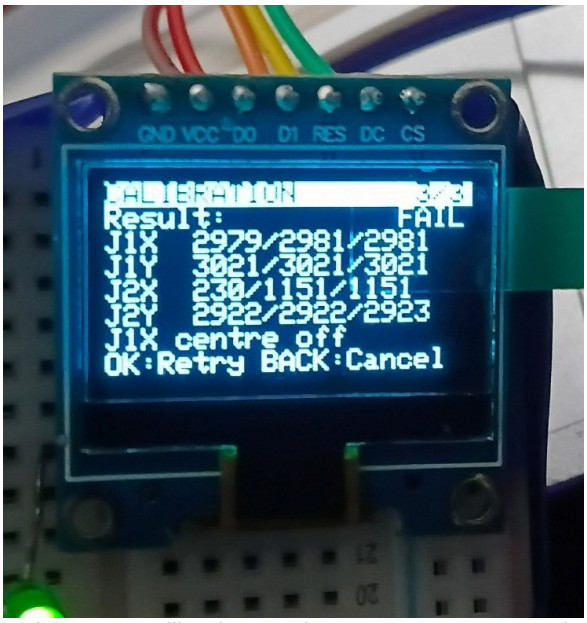


Figure 12: Calibration result screen: centres around 2980–3020 revealed the 5 V supply problem.

12.4 The second-stick investigation

The second stick caused most of the debugging effort: it moved by itself, one direction triggered a button, and another showed noise. A series of diagnostic tools was built into the firmware (wiring test, pin test, automatic wiring learning) and showed that its signals were reaching pins also used by buttons. Because the fault could not be isolated to a single wire in the breadboard, the design was simplified to one stick – which is also closer to a handheld layout with a D-pad on the left.



Figure 13: Xbox-style tester during the USB test: the right stick is stuck at −1.00 and the triggers show small values – the symptom of the faulty second stick.

12.5 Problems found and how they were solved

Problem	Cause	Solution (version)
Menus scrolled endlessly, stick values wrong at rest	Stick centre not calibrated; supply near 5 V	Auto-centre at boot, dead zone, simplified live calibration, "5V? Use 3V3!" warning (v1.1 – v2.0)
Screen went to sleep right after a press; loading animation invisible	A time stamp taken inside the loop was newer than the loop's own "now", giving a huge unsigned difference	Elapsed-time helper that cannot go negative, fresh time after event handling (v1.3)
Buttons "sometimes not working" over Bluetooth	A noisy axis generated a flood of reports and delayed button reports	Stick quantised to 63 steps, stick-only reports limited to 40/s, button changes sent first (v1.3)
Wrong button numbers in a tester or game	Different platforms number HID buttons differently	Platform tables (PC / Android) taken from the reference project, later Xbox mode (v1.5, v2.0)
Game ignored the controller on PC	Many PC games only read XInput (Xbox) controllers	Xbox (XInput) mode (v2.0)
Button order could not be saved	Confirmation used a key that was disabled on that screen; Center cancelled	Any "+" button saves, click cancels; assign screen saves with click or Center (v2.1)
Compile error: GamepadMode unknown	Older ESP32-BLE-Gamepad library	Update the library to 0.8.0+, or set USE_XINPUT to 0
Double cactus too hard at low speed	Jump too short / too quick	Gravity 500 px/s², launch speed 190 px/s, slower speed ramp (v1.2 – v1.4)

12.6 Offline logic checks

The firmware source was compiled with stand-in versions of the Arduino, display and Bluetooth libraries and driven by a test program. Checked this way: event generation for each control on each kind of screen; hat/D-pad direction building; Xbox and generic report contents for the three layouts and platforms (for example, with the "Face buttons" layout the Android mode produced buttons 1, 2, 4, 5, and the B5-equivalent menu key is never sent); the orientation wizard with swapped and reversed axes; the button-position wizard and its save/cancel behaviour; Snake rules; Pac-Man level completion; and the Dino jump timing window at several speeds. These checks do not replace testing on the real hardware, but they catch logic errors before flashing.

13. Troubleshooting Guide

Symptom	Likely cause	What to do
OLED stays dark; LED blinks fast (5 Hz)	OLED not found on SPI	Check the five SPI wires (D0→18, D1→23, RES→17, DC→16, CS→5), VCC and GND. The serial monitor prints the error.
Stick moves by itself or the cursor runs away	Not calibrated, noisy wire, or supply too high	Settings → Calibration → Calibrate range (keep the stick released first). Check the supply is 3V3 and the wires are short and firm.
"!" next to Calibration in Settings	No calibration/orientation saved, or stick centre reads high	Run Calibrate range and Orientation; if the screen says "5V? Use 3V3!" move the stick supply to 3V3.

Symptom	Likely cause	What to do
Stick directions swapped or reversed	Mounting or wiring differs	Settings → Calibration → Orientation, then recalibrate. For the game camera only, use Gamepad → Stick Y → Inverted.
A button does the wrong thing / roles mixed up	Button order differs from the default	Settings → Button positions (press Up, Down, Left, Right, Center), then press any "+" button to save.
Bluetooth shows CONNECTED but the host sees nothing	Host cached an old description	Remove the device from the host's Bluetooth list, pair again, press a button, check the TX counter and run Link test.
Controller not listed when scanning	Bluetooth screen not open, or already paired to another device	Open Main menu → Bluetooth first; switch the other host's Bluetooth off.
Game on PC does not react	Game only supports XInput	Use Mode: Xbox; for stubborn games add them to Steam and use Steam Input.
Racing: no throttle	Layout still D-pad	Gamepad → Layout → Racing (Up = RT, Down = LT, Left/Right = steering).
USB mode says "Host: waiting"	No program is talking to the port	Close the Serial Monitor, run usb_host.py, or type a character in the Serial Monitor at 115 200 baud.
Screen goes dark by itself	Display timeout	Settings → Display → Timeout → Off. The first press after the screen sleeps only wakes it.
Board restarted after changing the mode	Xbox ↔ generic needs a new descriptor	Normal. Pair again after the restart.
Everything strange after an update	Old settings	Settings → System → Factory reset (hold click 2 s).

14. Safety and Handling

- **3.3 V logic:** never apply 5 V to a GPIO pin. Power the stick and the OLED from 3V3.
- **Short circuits:** check the wiring before connecting USB, especially on a breadboard where rails can be accidentally bridged.
- **Static electricity:** touch a grounded object before handling the ESP32 and the OLED.
- **USB power:** the whole controller draws little current (OLED, LED and Bluetooth) and runs from a normal USB port. Use a data-capable cable.
- **Flash wear:** high scores and settings are written only when they change, so normal use does not wear the flash.
- **Photosensitivity:** the level-up animation flashes; the skip key (Right / click) ends it immediately.

15. Version History

Version	Main changes
v1.0	First complete firmware (first bring-up photos dated 4 November 2026): dual stick + five buttons, animated boot, OLED menus, calibration wizard, Bluetooth gamepad, USB serial stream, Dino game, LED modes, settings in flash.
v1.1	Pac-Man added; median ADC filtering and auto-centre at boot; simpler calibration; faster debouncing; Dino jump retuned.

Version	Main changes
v1.2	Nokia-style main menu, Games list, loading and level-up animations, live manual calibration, stick boxes on the Bluetooth screen, packet counter, Link test, new default mapping.
v1.3 – v1.4	Orientation wizard, input check, wiring and pin tests, fix for the timing underflow bug, Bluetooth rate limiting and 63-step quantisation, warnings moved into the Calibration menu.
v1.5 – v1.7	Buttons 1 and 2 also move the menu cursor; platform button numbering (PC / Android / PlayStation) taken from the reference project, Racing layout, hat switch, learn-wiring wizard, runtime pin table, J1/J2 clicks as menu keys.
v2.0	Architecture redesign: one stick + five-button "+" cluster. Xbox (XInput) mode, left-stick and D-pad targets, button-positions wizard. Second-stick diagnostics removed.
v2.1	Snake game; saving the button order fixed (any "+" button saves, click cancels); Assign buttons saves with Center or click; orientation result accepts the click.

16. Limitations and Future Work

16.1 Known limitations

- **No native USB gamepad:** the classic ESP32 offers USB serial only (section 9.3).
- **Bluetooth stays on:** once started from the menu, the Bluetooth radio keeps running until the next reset; leaving the Bluetooth screen only stops sending reports.
- **Xbox mode on the right stick Y axis:** the vertical direction convention follows the HID/Xbox standard (up is smaller); if a game shows it reversed, use Stick Y → Inverted.
- **One stick only:** the left-stick function is provided by buttons (full deflection). Analog left-stick control would need a second stick on dedicated ADC pins.
- **ADC accuracy:** the ESP32 ADC is not perfectly linear and slightly noisy; the calibration, median filter and dead zone compensate for most of it.
- **Power:** USB powered only; no battery or charger on the prototype.

16.2 Ideas for the future

- A version on an **ESP32-S3** board with native USB HID, so the controller works as a wired gamepad on phones and consoles with no program.
- Analog triggers on a second ADC input, or on the unused stick axis in racing layouts.
- Lithium battery with a charging board and a battery gauge on the OLED.
- More games (Breakout, Flappy-style, Tetris) using the existing games table; vibration motor for rumble.
- A PCB or a 3D-printed case with the "+" cluster and the stick positioned for both thumbs.
- Over-the-air firmware update and a phone app for configuration.

Appendix A – Complete GPIO Table

GPIO	Function	Direction	Notes
34	Stick VRx	Analog in	ADC1_CH6, input only
35	Stick VRy	Analog in	ADC1_CH7, input only
13	Stick click (SW)	Digital in	Internal pull-up, active LOW
27	Button 1 (default Up)	Digital in	Internal pull-up

GPIO	Function	Direction	Notes
26	Button 2 (default Down)	Digital in	Internal pull-up
25	Button 3 (default Left)	Digital in	Internal pull-up
33	Button 4 (default Right)	Digital in	Internal pull-up
32	Button 5 (default Center)	Digital in	Internal pull-up
18	OLED D0 (SCK)	SPI clock	Hardware SPI
23	OLED D1 (MOSI)	SPI data	Hardware SPI
5	OLED CS	Output	Chip select
16	OLED DC	Output	Data/command
17	OLED RES	Output	Reset
2	Status LED	Output	Via 1 kΩ to the LED; boot-strapping pin held LOW at start
3V3	OLED VCC, stick +V	Supply	3.3 V only
GND	All grounds	Supply	Common

Appendix B – Stored Settings (NVS)

Namespace `ctrl`. All values are validated at start-up.

Key	Size	Contents
ver	1 byte	Settings version (currently 4). An older version triggers an upgrade of the mapping.
map	6 bytes	Target number (0–20) for Up, Down, Left, Right, Center, Click.
bri	1 byte	Brightness level 1–5.
tmo	1 byte	Screen timeout: 0 Off, 1 = 30 s, 2 = 1 min, 3 = 5 min.
led	1 byte	LED mode: 0 Off, 1 On, 2 Status, 3 Activity.
pad	1 byte	Gamepad mode: 0 Xbox, 1 Generic PC, 2 Android.
yinv	1 byte	1 = stick Y inverted for the game.
hi, phi, shi	4 bytes each	Best scores: Dino Run (metres), Pac-Man, Snake.
cal	12 bytes	Stick calibration: minimum, centre, maximum for each of the two ADC channels (16-bit).
ori	4 bytes	Orientation: ADC channel and sign (±1) for the logical X and Y axes.
bpin	5 bytes	GPIO of the Up, Down, Left, Right and Center buttons.

Appendix C – Target Numbers

No.	Target	No.	Target	No.	Target
0	None	7	LT	14	Pad Down
1	A	8	RT	15	Pad Left
2	B	9	Select	16	Pad Right

No.	Target	No.	Target	No.	Target
3	X	10	Start	17	LS Up
4	Y	11	L3	18	LS Down
5	LB	12	R3	19	LS Left
6	RB	13	Pad Up	20	LS Right

The same numbers are used as bit positions in the USB serial mask (bit = number − 1).

Appendix D – Delivered Files and Credits

File	Purpose
ESP32_Game_Controller.ino	Complete firmware (v2.1) – one Arduino sketch.
usb_host.py	PC helper for the USB mode (pyserial, optional vgamepad).
joystick_design.png	Circuit diagram of the final design (Figure 2).
wirinfg_test_1.docx	Architecture note describing the "+" button cluster and the single stick.
ESP32 Dual-Joystick Controller – OLED Display Design / Final Documentation	Earlier design documents (Stage 1).
ESP_Bluetooth_Joystick.zip	Reference project (circuit image and sketch) used for the platform button tables.
goku-dragon-ball-level-up-anime-gif.h	Level-up animation frames.
super_sian_display.ino	Loading animation frames.
Photographs and tester screenshots	Figures 4, 8, 11, 12 and 13 of this document.

Credits. Firmware and design: HARRISH YESURAJ P. Libraries: Adafruit GFX and SSD1306, NimBLE-Arduino, ESP32-BLE-Gamepad by lemmingDev. Reference project: "Tech at Home". The animations were supplied by the developer.

Appendix E – Glossary

Term	Meaning
ADC	Analog-to-digital converter; turns the stick voltage into a number 0 – 4095.
BLE	Bluetooth Low Energy.
D-pad / hat	The four-direction pad of a gamepad; sent as one "hat switch" value with eight directions.
Dead zone	A small region around the centre in which the stick counts as released.
HID	Human Interface Device: the standard way keyboards, mice and gamepads talk to a host.
NVS / Preferences	The ESP32's non-volatile storage in flash memory.
OLED	Organic light-emitting diode display; here 128×64 pixels, one colour.
SPI	Serial Peripheral Interface; the 4-wire bus to the display.
XInput	The Microsoft controller interface used by Xbox controllers and most PC games.
Median filter	Takes the middle value of several readings, which ignores single bad readings.